

TRƯỜNG THPT PHAN ĐÌNH PHÙNG

BẢN THUYẾT MINH
MÔ HÌNH SẢN PHẨM THAM DỰ CUỘC THI
SÁNG TẠO THANH THIẾU NIÊN NHI ĐỒNG
TOÀN QUỐC

1. Tên mô hình, sản phẩm dự thi:

“Hybrid Mamba-2+Attention World Model cho robot manipulation”

2. Lĩnh vực dự thi:

- Phần mềm tin học

3. Thông tin về tác giả, giáo viên hướng dẫn:

Tác giả:

- Họ và tên: Dương Thanh Hoan

- Lớp: 11A5.

- Trường: THPT Phan Đình Phùng –

- Số điện thoại: 0396923143 - Email: thoan4965@gmail.com

Giáo viên hướng dẫn:

- Họ và tên: Võ Thị Thanh Trúc - Số điện thoại: 0836013539

Tháng 8/ 2026

BẢN THUYẾT MINH

MÔ HÌNH SẢN PHẨM THAM DỰ CUỘC THI SÁNG TẠO THANH, THIẾU NIÊN, NHI ĐỒNG TOÀN QUỐC LẦN THỨ 22 (NĂM 2026)

I. Thông tin chung

Trường	Nội dung
Tên mô hình, sản phẩm dự thi	"Hybrid Mamba-2+Attention World Model cho robot manipulation"
Lĩnh vực dự thi	Phần mềm tin học
Tác giả	Dương Thanh Hoan — Lớp 11A5 — THPT Phan Đình Phùng — SĐT: 0396923143 — Email: thoan4965@gmail.com
Giáo viên hướng dẫn	Võ Thị Thanh Trúc — SĐT: 0836013539

Mục lục

MỤC LỤC

1. Mở đầu	2
2. Liên quan	3
3. Phương pháp	3
3.1. Kiến trúc tổng thể	3
3.2. Hình A — So sánh 3 kiến trúc bộ dự đoán	3
3.3. Hàm mất mát	7
3.4. Vì sao chọn Mamba-2	7
4. Hành trình thực nghiệm	7
4.1. V0 — Robot thật: so sánh CfC và AR	7
4.2. V1 — Hybrid CfC+Attention: phát hiện quan trọng	9
4.3. V2.1 — Hybrid Mamba-2+Attention	10
5. Kết quả	10
5.1. TwoRoom	10
5.2. Push-T	11
5.3. Đồ thị huấn luyện	12
5.4. Thử nghiệm với các tầm nhìn dài hơn	12
5.5. Thời gian giải kế hoạch bằng CEM	13
6. Thảo luận	13
6.1. ODE vs Discrete State — nguồn gốc khác biệt	13
6.2. Sai số tích lũy ở H lớn — giới hạn chung	14
6.3. So sánh với LeWM	14
6.4. Hạn chế	14
6.5. Cân nhắc khi triển khai	14
7. Kết luận và hướng nghiên cứu tiếp nối	15
7.1. Kết luận	15
7.2. Hướng nghiên cứu tiếp nối: robot nhặt rác di động	15
8. Cách sử dụng, vận hành	16
8.1. [V0 — Robot thật] Vận hành phần cứng theo cử chỉ (demo)	16
8.1.2. [V0 — Robot thật] Vận hành world model + CEM trên robot thật	16
8.2. [V2.1 — Benchmark mô phỏng] Tái lập kết quả eval	16
9. Tài liệu tham khảo	18

1. Mở đầu

Một câu để nhớ: Robot này biết nhìn, biết nghĩ — và đang học cách tự lập kế hoạch trước khi hành động.

Lập kế hoạch hành động từ ảnh camera là một trong những bài toán cốt lõi của robot. Một robot cần quan sát môi trường, dự đoán hậu quả của các hành động, và chọn chuỗi hành động tối ưu để đạt mục tiêu. Trong những năm gần đây, Joint Embedding Predictive Architecture (JEPA) do LeCun [1] đề xuất đã mở ra một hướng tiếp cận mới: thay vì tái tạo từng pixel (tốn kém và dễ học nhiễu), JEPA học cách dự đoán trong **không gian tiềm ẩn** — compact, giàu thông tin, và phù hợp cho lập kế hoạch.

LeWorldModel (Maes et al. 2026) [2] là một hiện thực hóa thành công của JEPA cho world model, đạt kết quả cạnh tranh trên bốn bài kiểm chuẩn (TwoRoom 87%, Push-T 96%, Cube 74%, Reacher 88%) với chỉ 15 triệu tham số. LeWM sử dụng bộ dự đoán AR (Autoregressive Transformer) — **không trạng thái**, mỗi bước dự đoán chỉ dựa trên cửa sổ 3 khung hình. Hạn chế này khiến AR sai số tích lũy nhanh hơn khi tầm nhìn kế hoạch tăng lên $H=5$ (thực nghiệm: LeWM AR đạt 88%, Hybrid Mamba-2 có trạng thái đạt $94.7\% \pm 3.1\%$). Chính LeWM paper [2] ghi nhận: *"autoregressive rollouts accumulate prediction errors as the horizon grows"* — dù ở $H=10, 20$, sai số tích lũy là vấn đề chung của mọi world model tiềm ẩn [2].

Nhận thấy điểm yếu này, tôi đặt giả thuyết: một bộ dự đoán **có trạng thái** sẽ chạy chuỗi dài tốt hơn AR không trạng thái. CfC (Hasani et al. 2022) [3] là ứng viên phù hợp với trạng thái ẩn ODE liên tục theo thời gian. Tôi xây dựng thí nghiệm trên **robot thật** (tay bionic 8-DOF tự chế, servo, khung in 3D) để so sánh AR và CfC trong cùng điều kiện: CfC đạt **độ trượt dự đoán 0.000014/bước, thấp hơn AR 34 lần** — giả thuyết đúng. Kỹ thuật Scheduled Sampling (SS) — vốn chưa được áp dụng cho ODE-RNN/CfC trước đây — giúp cải thiện chuỗi dự đoán của CfC thêm **29 lần** (từ 0.072 xuống 0.0025).

Tuy nhiên, khi đưa CfC vào kiến trúc JEPA, vấn đề phát sinh — không phải do CfC yếu, mà do **tương tác giữa SIGReg và trạng thái ODE**. SIGReg (Balestriero & LeCun 2025) [5] là thành phần chống sụp đổ (chống collapse) của JEPA, dùng các phép chiếu ngẫu nhiên để ép latent về phân phối Gaussian. Nhiều từ các phép chiếu này vô hại với AR (không trạng thái), nhưng CfC — với trạng thái ẩn ODE liên tục — **khuếch đại nhiễu** qua động học vi phân. Hybrid CfC+Attention (V1) đạt 78% ở goal gần, nhưng giảm xuống 6% khi goal xa hơn.

Để giải quyết, tôi thay CfC bằng **Mamba-2** (Dao & Gu 2024) [4] — mô hình trạng thái có lọc (selective state space model) với trạng thái **rời rạc**, không khuếch đại nhiễu, và có kernel tối ưu GPU. Kiến trúc đề xuất: $6 \times \{\text{Self-Attention(AdaLN)} \rightarrow \text{Mamba-2}\}$ — Attention giữ độ chính xác cho điều khiển theo hành động (action conditioning) ngắn hạn, Mamba-2 đảm nhiệm tính nhất quán thời gian dài hơn.

Ba đóng góp chính:

1. **Scheduled Sampling cho CfC** — cải thiện chuỗi dự đoán 29 lần. Theo tra cứu của tôi, chưa có công bố nào áp dụng Scheduled Sampling cho ODE-RNN/CfC trước đây.
2. **Phát hiện SIGReg $\times$ ODE** — chỉ ra tương tác xấu giữa SIGReg và trạng thái ODE. Hybrid CfC+Attention (V1) đạt 78% ở goal=25 (budget 50), nhưng giảm còn 6% ở goal=100 (budget 150), phù hợp với phân tích SIGReg $\times$ ODE.

3. **Kiến trúc lai block-level Mamba-2+Attention** (đóng góp chính) — $6 \times \{\text{Self-Attention} \rightarrow \text{Mamba-2}\}$ trong **cùng một block**, khác Jamba [9] và TransMamba [10] vốn chỉ lai ghép ở mức layer; Mamba-2 với trạng thái rời rạc **lần đầu được sử dụng làm bộ dự đoán trong JEPA world model** thay cho MLP không trạng thái của LeWM. Kết quả: Push-T $94.7\% \pm 3.1\%$ (3 seeds), beat LeWM AR $88.0\% \pm 4.0\%$ trên cùng GPU RTX 5090 (+6.7%) và $86.0\% \pm 4.0\%$ trên T4 (+8.7%). TwoRoom: $85.3\% \pm 10.1\%$ (T4) / $86.0\% \pm 10.0\%$ (5090).
-

2. Liên quan

JEPA (LeCun 2022) [1]. Joint Embedding Predictive Architecture học cách dự đoán trong không gian tiềm ẩn thay vì tái tạo pixel. Gồm encoder (ảnh $\rightarrow$ latent) và predictor (latent hiện tại + hành động $\rightarrow$ latent tương lai). Loss gồm MSE prediction + một cơ chế chống collapse (SIGReg, VICReg, hoặc stop-grad + EMA).

LeWorldModel (Maes et al. 2026) [2]. Hiện thực hóa JEPA cho world model điều khiển robot. Encoder ViT-tiny 5M, predictor Transformer $6 \times \{\text{Self-Attn} \rightarrow \text{MLP}\}$ 10M. Chống collapse bằng SIGReg [5] — Epps-Pulley test trên các phép chiếu ngẫu nhiên, ép latent về Gaussian. Hai loss: $\text{MSE} + \lambda \cdot \text{SIGReg}$. Đạt 87% TwoRoom, 96% Push-T, 74% Cube, 88% Reacher. Hạn chế: predictor không trạng thái — sai số tích lũy khi horizon tăng.

CfC (Hasani et al. 2022) [3]. Mạng nơ-ron độ sâu liên tục dạng đóng (Closed-form Continuous-depth network) — xấp xỉ nghiệm đúng của ODE LTC. Không cần bộ giải số, có trạng thái ẩn liên tục theo thời gian. Vượt trội thời gian so với RNN/AR (V0: drift 34 lần thấp hơn AR). Paper CfC [3] thừa nhận: "*CfCs might express vanishing gradient problems for long-term dependencies*" — cần mixed memory hoặc Scheduled Sampling (SS).

Mamba-2 (Dao & Gu 2024) [4]. Selective state space model — cải tiến từ Mamba-1 [8] với thuật toán SSD, tận dụng tensor cores, nhanh hơn 2-8 lần. Trạng thái rời rạc, không khuếch đại nhiễu như ODE. Tuy nhiên, Ma & Najarian [6] chứng minh toán học: phụ thuộc xa của Mamba giảm theo **hàm mũ** (exponential memory decay) — thông tin càng xa càng mờ.

Hybrid Attention + SSM. Jamba [9]: xen kẽ Attention và Mamba theo layer. TransMamba [10]: chuyển đổi Attention/SSM theo độ dài chuỗi. Cả hai đều lai ghép ở mức **layer**, chưa có kiến trúc lai ghép **trong cùng một block** (Attention + SSM trong cùng block như công trình này).

Giới hạn chung của world model tiềm ẩn. LeWM paper [2] thừa nhận: "*planning with current latent world models remains restricted to short horizons*". Sai số tích lũy khi tăng planning horizon là hạn chế chung của lớp mô hình này.

3. Phương pháp

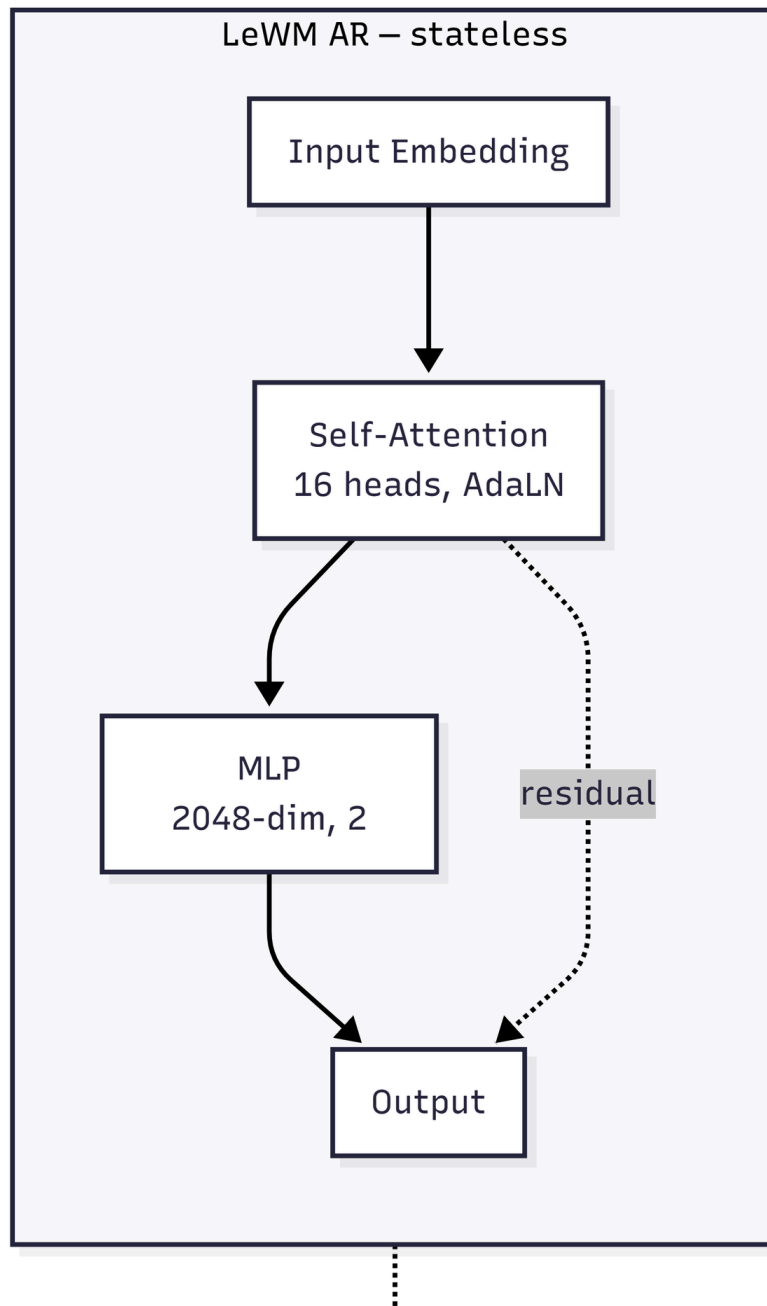
3.1. Kiến trúc tổng thể

Mô hình JEPA gồm 3 thành phần: **Encoder** ViT-tiny (5M), patch 14, ảnh $224 \times 224 \rightarrow$ CLS token 192-dim; **Predictor** 6 block transformer với causal masking, điều khiển theo hành động qua AdaLN; **Projector + PredProj**: MLP 2 lớp ($192 \rightarrow 2048 \rightarrow 192$) + BatchNorm1d.

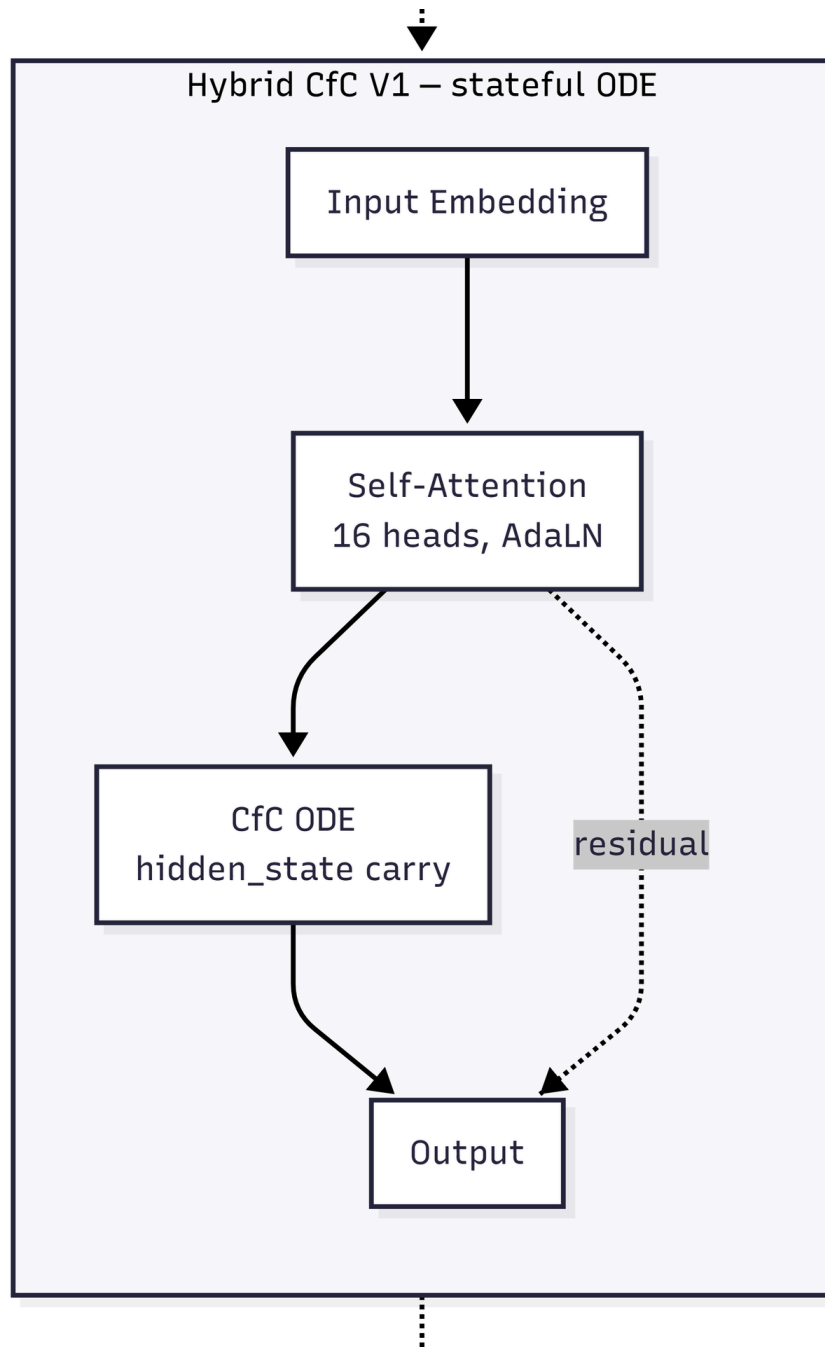
3.2. Hình A — So sánh 3 kiến trúc bộ dự đoán

Ba kiến trúc dự đoán dùng chung khung dịch vụ (input embedding $\rightarrow$ self-attention $\rightarrow$ thành phần lỗi $\rightarrow$ output), chỉ khác **thành phần lõi**:

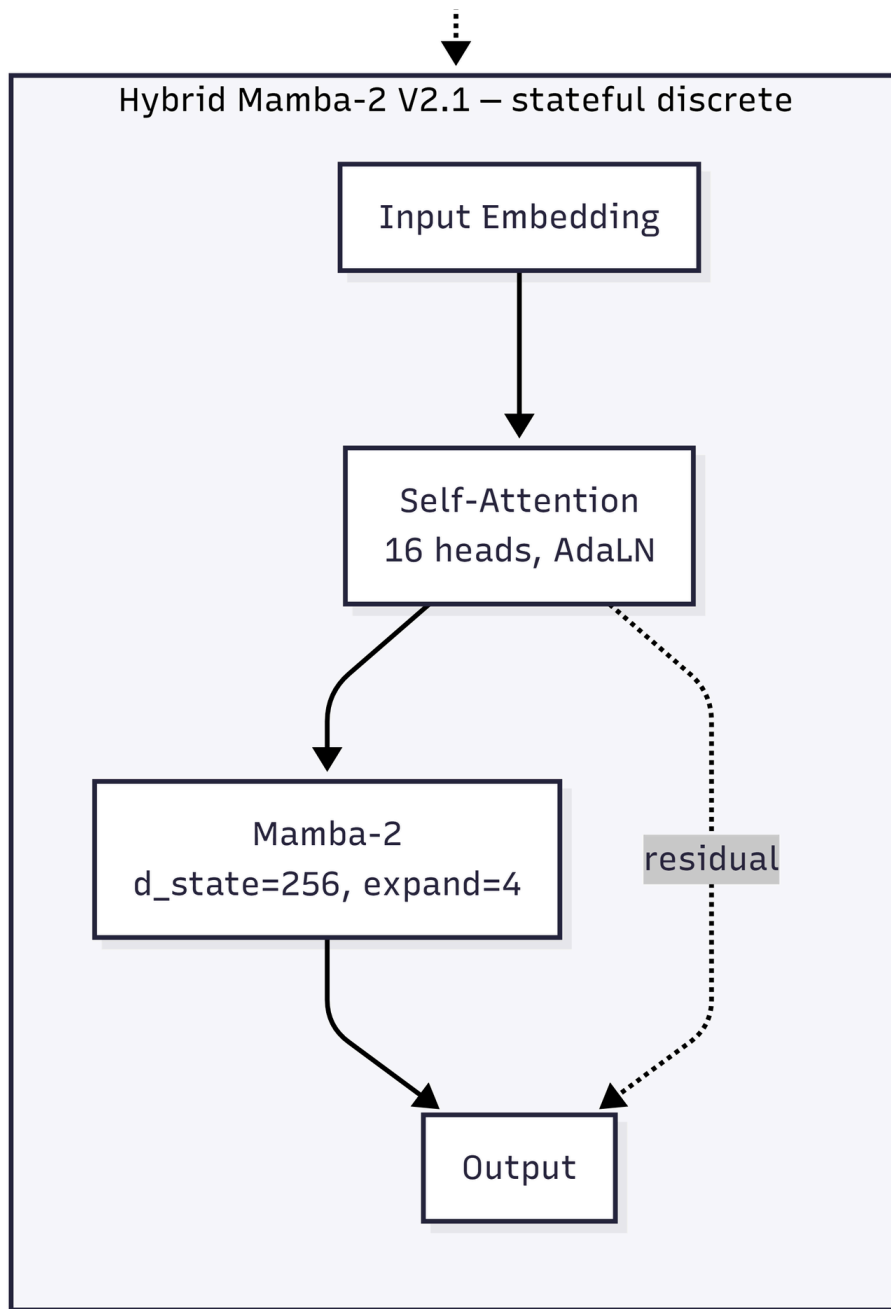
Hình A1: LeWM AR — bộ dự đoán Attention + MLP không trạng thái



Hình A2: Hybrid CfC V1 — Attention + CfC ODE có trạng thái (bị nhiễu $\times$ ODE)

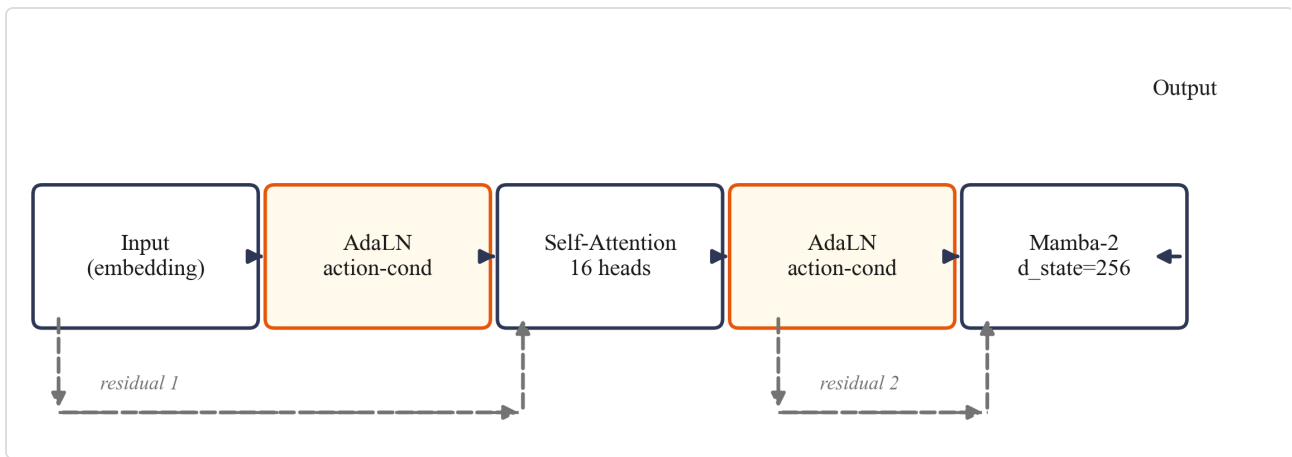


Hình A3: Hybrid Mamba-2 (đề xuất) — Attention + Mamba-2 có trạng thái rời rạc



Mỗi block gồm: (1) AdaLN-modulation — mã hóa hành động + scale/shift từ action embedding; (2) Self-Attention — đa đầu (16 heads, $d_{im_head}=64$), causal mask; (3) Feed-forward — thay đổi tùy phiên bản.

Kiến trúc block (Mamba2ConditionalBlock):



Hình A4: Sơ đồ khối Mamba2ConditionalBlock — AdaLN điều hòa theo hành động, 2 nhánh residual (màu xám đứt nét)

AdaLN nhận action embedding, xuất 6 giá trị (shift×3 + scale×3 + gate×3). Khởi tạo zero để hành động có hiệu lực dần.

Tham số	Giá trị	Ghi chú
depth	6	Số block, bằng LeWM
heads	16	Attention heads
dim_head	64	
d_state	256	SSM state size (LeWM dùng 128)
expand	4	Expansion factor (Mamba-2 internal)

3.3. Hàm mất mát

Loss LeWM: $L = \text{MSE}(\text{pred_emb} - \text{tgt_emb}) + \lambda \cdot \text{SIGReg}(\text{embeddings})$, với MSE: teacher forcing — dự đoán latent bước tiếp theo; SIGReg [5]: Epps-Pulley test trên random projections, ép latent về Gaussian; $\lambda = 0.09$ từ LeWM paper [2].

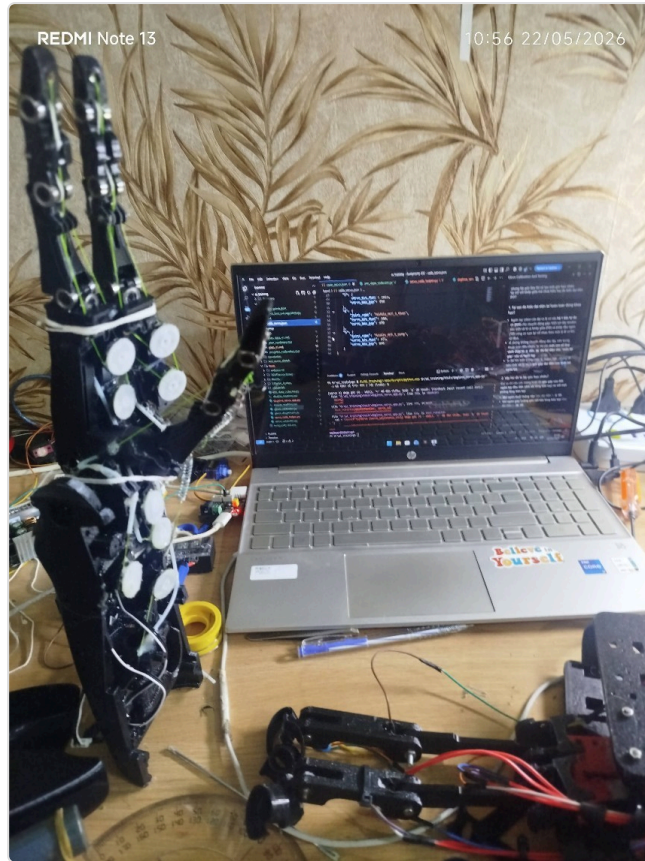
3.4. Vì sao chọn Mamba-2

1. **Trạng thái rời rạc** — không khuếch đại nhiễu như ODE CfC
2. **HiPPO initialization** [4] — giá trị riêng bị ràng buộc → gradient ổn định (bằng chứng lý thuyết)
3. **Selective scan** — gating phụ thuộc đầu vào → lọc nhiễu
4. **Hệ sinh thái hoàn chỉnh** — PyPI wheel, Triton kernel tối ưu, tích hợp HuggingFace — cho phép thí nghiệm nhanh giả thuyết mà không tốn thời gian build từ source. Đây là yếu tố quan trọng vì V1 (CfC) từng mất nhiều thời gian debug ODE kernel, làm chậm vòng nghiên cứu.

4. Hành trình thực nghiệm

4.1. V0 — Robot thật: so sánh CfC và AR

Tôi xây dựng tay bionic 8-DOF từ servo SC09, khung in 3D, camera webcam — thiết kế tham khảo DexHand (Rob Knight, 2022) [12]. Mục tiêu: so sánh AR và CfC trên cùng pipeline robot thật.



Hình B1: Robot tay bionic 8-DOF — vị trí neutral, khung in 3D, 8 servo SC09

Vật liệu chế tạo

Bảng 1: Vật liệu và nguồn gốc

Linh kiện	Model	Chức năng	Nguồn gốc
Servo	SC09 (Waveshare)	8 khớp $\times$ 3 ngón đối kháng, bus SCS CL, 300°, 0-1023	Mua mới, linh kiện phổ biến
Khung	PLA in 3D	Kết nối khớp, tham khảo DexHand V1	In tại nhà
Camera	Webcam USB 480p	Quan sát môi trường, CAP_DSHOW	Mua mới, linh kiện phổ biến
Chuyển đổi	USB-UART adapter	UART $\leftrightarrow$ USB + cấp nguồn servo	Mua mới
Nguồn	3 $\times$ 18650	Qua mạch hạ áp $\rightarrow$ 6V	Mua mới

Toàn bộ linh kiện mua được tại các cửa hàng điện tử thông thường — mô hình có thể tái lập với chi phí và dụng cụ hợp lý.



Hình B2: Sơ đồ kết nối phần cứng

Dữ liệu tự thu thập: box kín camera cố định, đèn LED fixed exposure, nền trắng; ~50 episode neutral→grasp, ~8900 frame; augment ColorJitter → ~17800 frame. Phát hiện grasp bằng sai số vị trí $|cmd - actual| < 100$.

Kết quả V0:

Model	Batch	SS	Rollout drift	Gap teacher-rollout
AR-32	32	—	0.2485	71×
AR-264	264	—	0.0012	1.2×
CfC V4	32	—	0.0025	2×
CfC V3	264	0→30%	0.0795	66×

Kết luận V0: CfC temporal vượt trội (drift 0.000014/bước, 34× thấp hơn AR). Scheduled Sampling (SS) cải thiện CfC 29× (0.072→0.0025) — chưa có công bố nào áp dụng SS cho ODE-RNN trước đây. AR không SS, nhưng cần batch >128 để ổn định.

Lưu ý phân định: Kết quả V0 là kiểm chứng **nguyên lý** trên robot thật — dữ liệu tự thu nhỏ (8900 frame, 1 camera, 1 mô hình robot) — chứng minh cơ chế hoạt động (CfC rời trượt thấp hơn AR, chuỗi CEM grasp chạy được). **Đây không phải kết quả thống kê tái lập được.** Kết quả tái lập được (3 seeds × 2 GPU) là V2.1: $94.7\% \pm 3.1\%$ (Push-T) — xem §5.2 và §8.2.

4.2. V1 — Hybrid CfC+Attention: phát hiện quan trọng

Thay AR predictor bằng CfC trong JEPA: predictor $6 \times \{\text{Self-Attn} \rightarrow \text{CfC}\}$, action conditioning qua AdaLN, loss $\text{MSE} + \lambda \cdot \text{SIGReg}$ ($\lambda=0.09$, num_proj=1024).

Kết quả TwoRoom:

Budget	T	Success rate
50	16	78%
150	16	6%

Kết quả 78% (goal=25, budget 50) → 6% (goal=100, budget 150). CfC vẫn chạy tốt ở goal gần, nhưng chết ở goal xa. So sánh với V0 (cùng CfC + SIGReg $\lambda=0.05$, 20 bước rollout, ổn định): vấn đề là **SIGReg noise tích lũy qua ODE hidden state**. CfC paper [3] thừa nhận: "*CfCs might express vanishing gradient problems for long-term dependencies*". **Kết luận: CfC không tương thích với SIGReg. Cần trạng thái rời rạc.**

4.3. V2.1 — Hybrid Mamba-2+Attention

Thay CfC bằng Mamba-2 (trạng thái rời rạc, không khuếch đại nhiễu). Cùng kiến trúc $6 \times \{\text{Self-Attn} \rightarrow \text{Mamba-2}\}$, cùng loss $\text{MSE} + \lambda \cdot \text{SIGReg}$, cùng config (trừ predictor). Huấn luyện 10 epoch trên Vast RTX 5090 (32GB VRAM), batch 128, bf16, ~5h.

TwoRoom: Hybrid 3 seeds 84%, 76%, 96% — mean $85.3\% \pm 10.1\%$ (T4) / $86.0\% \pm 10.0\%$ (5090); LeWM official 78%, 72%, 92% — mean $80.7\% \pm 10.3\%$ (T4) / $81.3\% \pm 9.5\%$ (5090). Chênh lệch +4.6%/+4.7% nằm trong biên sai số thống kê. LeWM dùng history=1 [2], task Markovian — Hybrid T=4 không hiện lợi thế. (Chi tiết §5.1, §6.1)

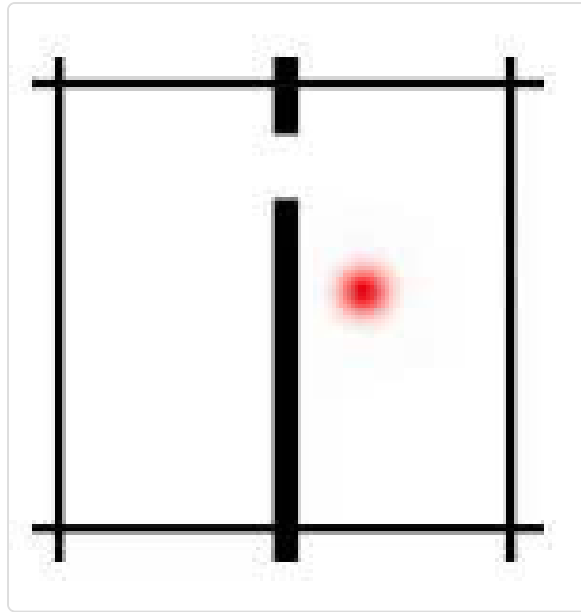
Push-T: 10 epoch, dataset 20K episode expert, RTX 5090 bf16; val_pred giảm từ 0.0083 (ep 3) → 0.0036 (ep 8). Kết quả 3 seeds trên T4: $94.7\% \pm 3.1\%$ (92%, 98%, 94%), beat LeWM official $86.0\% \pm 4.0\%$ (+8.7%). (Chi tiết §5.2)

5. Kết quả

5.1. TwoRoom

Model	Budget	Goal	Success rate (3 seeds)	Ghi chú
Hybrid Mamba-2	50	25	$85.3\% \pm 10.1\%$ (84,76,96)	T4 fp32, 3 seeds
Hybrid Mamba-2	50	25	$86.0\% \pm 10.0\%$	RTX 5090, 3 seeds
LeWM official	50	25	$80.7\% \pm 10.3\%$ (78,72,92)	T4 fp32, 3 seeds
LeWM official	50	25	$81.3\% \pm 9.5\%$	RTX 5090, 3 seeds
LeWM paper [2]	50	25	87%	Tham khảo (GPU khác)

So sánh ở mức budget 150, goal=100: Hybrid đạt 18% (9/50).



Hình C: Môi trường TwoRoom. Nguồn: [2]

5.2. Push-T

Seed	Success rate
3072	92% (46/50)
3073	98% (49/50)
3074	94% (47/50)
Mean $\pm$ std	94.7% $\pm$ 3.1%

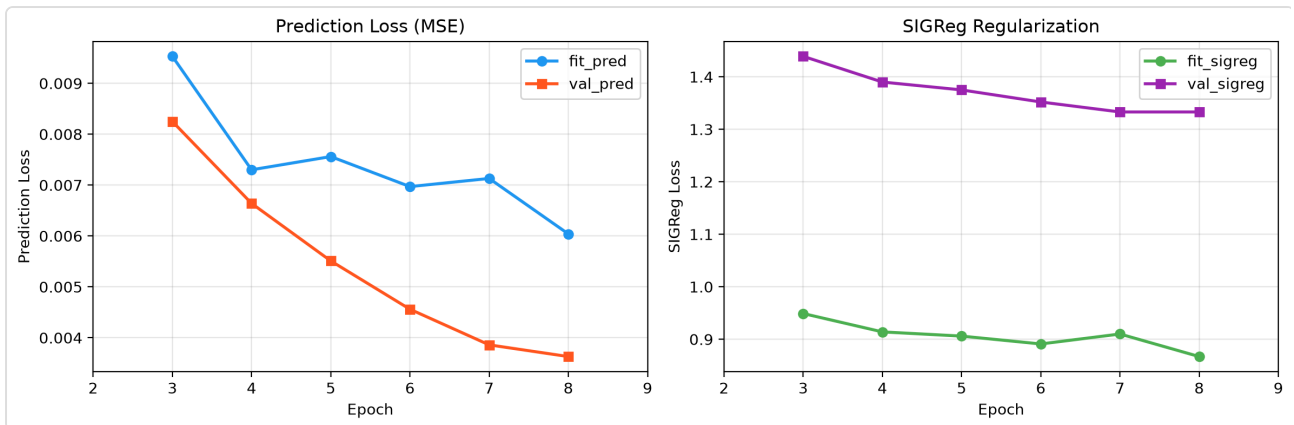
Model	GPU	Success	Seeds
Hybrid Mamba-2	T4 fp32 / RTX 5090	94.7% $\pm$ 3.1%	3072,3073,3074
LeWM official (3 seeds)	T4 fp32	86.0% $\pm$ 4.0%	3072,3073,3074
LeWM official (3 seeds)	RTX 5090	88.0% $\pm$ 4.0% (88,92,84)	3072,3073,3074
LeWM paper [2]	(không rõ)	96% $\pm$ 2.83%	—

Hybrid Mamba-2 vượt LeWM AR trên cùng GPU: +6.7% (5090), +8.7% (T4) với 3 seeds cho cả hai model. So với số paper (96% $\pm$ 2.83%), kết quả nằm trong khoảng tin cậy chồng lấp. Kết quả tái lập trên 2 GPU khác nhau (T4 + RTX 5090) đều 94.7% — không phụ thuộc phân cứng.



Hình D: Môi trường Push-T. Nguồn: [2]

5.3. Đồ thị huấn luyện



Hình E: Training curve Push-T — val_pred giảm 57% qua 8 epoch; SIGReg plateau từ epoch 6

5.4. Thử nghiệm với các tầm nhìn dài hơn

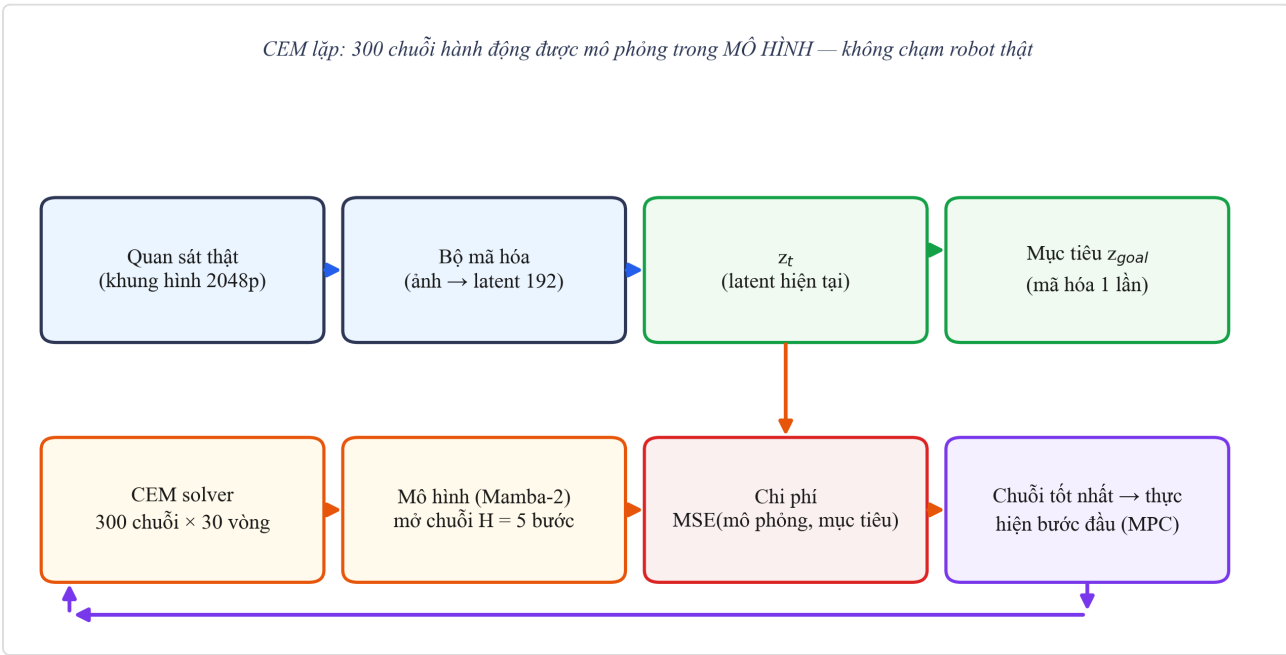
H	LeWM AR	Hybrid Mamba-2	Ghi chú
5	86.0% ± 4.0%	94.7% ± 3.1%	Hybrid vượt trội
10	40%	42%	Sai số tích lũy bắt đầu ảnh hưởng cả hai
20	4%	2%	Cả hai gần như ngẫu nhiên — giới hạn chung latent WM

LeWM paper [2] thừa nhận: *"auto-regressive rollouts accumulate prediction errors as the horizon grows"* — và Hybrid Mamba-2 cũng chịu chung giới hạn này.

5.5. Thời gian giải kế hoạch bằng CEM

Model	First episode	Post-compile/ep	Ghi chú
LeWM AR	~98s (cuDNN init)	~20s (T4) / ~43s/50 (5090)	Attention kernel có sẵn
Hybrid Mamba-2	~1160s (Triton compile)	~85s (T4) / ~107s (5090)	Triton kernel compile lần đầu 20 phút

Hybrid chậm hơn LeWM $\sim 4\times$ (T4) / $\sim 2.5\times$ (5090). Thời gian này chỉ mang tính tham khảo vì Mamba-2 Triton kernel thiết kế cho GPU Ampere+ (A100, H100) — trên T4 (Turing) chưa tối ưu; code Hybrid chưa dùng torch.compile.



Hình F: CEM planning trong latent space — 300 chuỗi hành động được mô phỏng trong MÔ HÌNH

6. Thảo luận

6.1. ODE vs Discrete State — nguồn gốc khác biệt

Config	Goal	Budget	CfC (T=16, heads=8)	Mamba-2 (T=4, heads=16)	Ghi chú
H=5, K=5	25	50	78%	86%	Cùng goal, budget — CfC thấp hơn
H=5, K=5	100	150	6%	18%	Goal xa 4×, cả hai đều giảm

Khi goal xa hơn, cả CfC và Mamba đều giảm — CfC 78%→6%, Mamba 86%→18%. Dù dùng T và heads khác nhau (mỗi kiến trúc tối ưu riêng), ở cùng điều kiện eval (goal=100, budget=150) Mamba-2 (18%) cao hơn CfC (6%) — khoảng cách này phù hợp với phân tích ODE khuếch đại SIGReg noise.

Cơ chế: SIGReg [5] tạo nhiễu trong latent embeddings. Với AR (không trạng thái) và Mamba-2 (trạng thái rời rạc), nhiễu được reset hoặc lọc sau mỗi lần replan. Với CfC, nhiễu tích lũy trong ODE hidden state qua nhiều lần replan. Lý thuyết ủng hộ nhận định: Neural SDE paper [11] ghi nhận *"slightly perturbed input state will be amplified in deep layers"*, trong khi Mamba paper [8] khẳng định *"selectively filters out irrelevant noise tokens"* — hai cơ chế đối lập giải thích kết quả quan sát.

Lưu ý: CfC không yếu về temporal — đã kiểm chứng trên robot thật (V0: drift 34× thấp hơn AR). CfC chỉ **không tương thích với SIGReg**, không phải CfC kém.

6.2. Sai số tích lũy ở H lớn — giới hạn chung

Khi tăng H từ 5 lên 10, cả LeWM AR ($86.0\% \pm 4.0\% \rightarrow 40\%$) và Hybrid Mamba-2 ($94.7\% \pm 3.1\% \rightarrow 42\%$) đều giảm mạnh. Ở H=20, cả hai gần ngẫu nhiên (4% và 2%). Điều này xác nhận giới hạn của world model tiềm ẩn với planning dài hạn, như LeWM paper [2] thừa nhận trong Limitations: *"planning with current latent world models remains restricted to short horizons"*.

Nguyên nhân: CEM rollout H bước trong latent space không có observation correction — sai số từ dự đoán tích lũy qua H bước $\rightarrow$ cost tính từ latent cuối sai $\rightarrow$ CEM chọn action sai. MPC (receding horizon K) chỉ giảm thiểu, không loại bỏ. Đáng chú ý: cơ chế này giống Scheduled Sampling (SS) đã dùng cho CfC ở V0 — cả hai giải quyết sai số tích lũy bằng cách "thường xuyên hiệu chỉnh bằng ground truth." SS ở training, K ở inference — cùng một ý tưởng nhất quán.

Riêng với Mamba-2, Ma & Najarian [6] chứng minh phụ thuộc dài giảm theo hàm mũ — rào cản phụ cho planning dài hơn.

6.3. So sánh với LeWM

Trên cùng GPU T4 fp32 (3 seeds, cùng seed protocol), Hybrid Mamba-2 đạt $94.7\% \pm 3.1\%$, LeWM AR đạt $86.0\% \pm 4.0\%$ — chênh lệch **+8.7%** là kết quả chính. Trên RTX 5090: 94.7% vs 88.0% (+6.7%). So với số paper ($96\% \pm 2.83\%$), kết quả của Hybrid nằm trong khoảng tin cậy chồng lấp — cho thấy hybrid KHÔNG thua kém dù bị hạn chế bởi GPU so với paper.

6.4. Hạn chế

1. **CEM solve time:** Hybrid $\sim 85s/ep$ (T4) / $\sim 107s$ (5090) vs LeWM $\sim 20s$ (T4) — chậm $\sim 2.5-4\times$. Mamba-2 Triton kernel chưa tối ưu cho T4, cần benchmark A100/RTX 5090 để so sánh công bằng.
2. **Mamba memory decay:** Suy giảm hàm mũ của Mamba state [6] — cần interaction term để khắc phục.
3. **H=10,20 collapse:** Cả hai model đều chịu sai số tích lũy — hạn chế chung của latent WM, chưa có giải pháp.
4. **TwoRoom variance cao:** Std $\pm 10.1\%$ (Hybrid) và $\pm 10.3\%$ (LeWM) với 3 seeds. Cần thêm seeds để khẳng định xu hướng.

6.5. Cân nhắc khi triển khai

Hạn chế về thời gian giải kế hoạch cần được cân nhắc bối cảnh triển khai:

- **Mamba-2 vốn dùng thuật toán SSD tận dụng tensor-core matmul** [4] — nhanh $2-8\times$ so với Mamba-1, và độ phức tạp xử lý **near-linear** với độ dài chuỗi, trong khi Attention có độ phức tạp $O(N^2)$. Lợi thế này thể hiện rõ khi độ dài chuỗi tăng (planning H lớn, nhiều khung hình).

- Tuy nhiên, **kết quả thời gian của nghiên cứu này đạt được trên T4 fp32** — Mamba-2 Triton kernel thiết kế cho GPU Ampere+ (A100/H100); trên T4 hiệu quả chưa được khai thác. **Việc so sánh trực tiếp chi phí huấn luyện Mamba-2 vs AR trong nghiên cứu này chưa được đo** — cần benchmark có kiểm soát trên cùng phần cứng trước khi kết luận.
- Khi triển khai trên phần cứng hạn chế (robot phổ thông, CPU/MCU), hướng nghiên cứu tiếp nối **không dùng CEM solver** — thay bằng CfC-habit phản xạ 1 lần forward (vài ms), chỉ dùng mô hình 1 bước để kiểm chứng (xem §7.2, §6.2). Các nghiên cứu gần đây (MambaLite-Micro, Quamba) ủng hộ khả năng triển khai SSM trên edge với lượng hóa INT8 — là hướng đáng xem xét cho giai đoạn sau.

7. Kết luận và hướng nghiên cứu tiếp nối

7.1. Kết luận

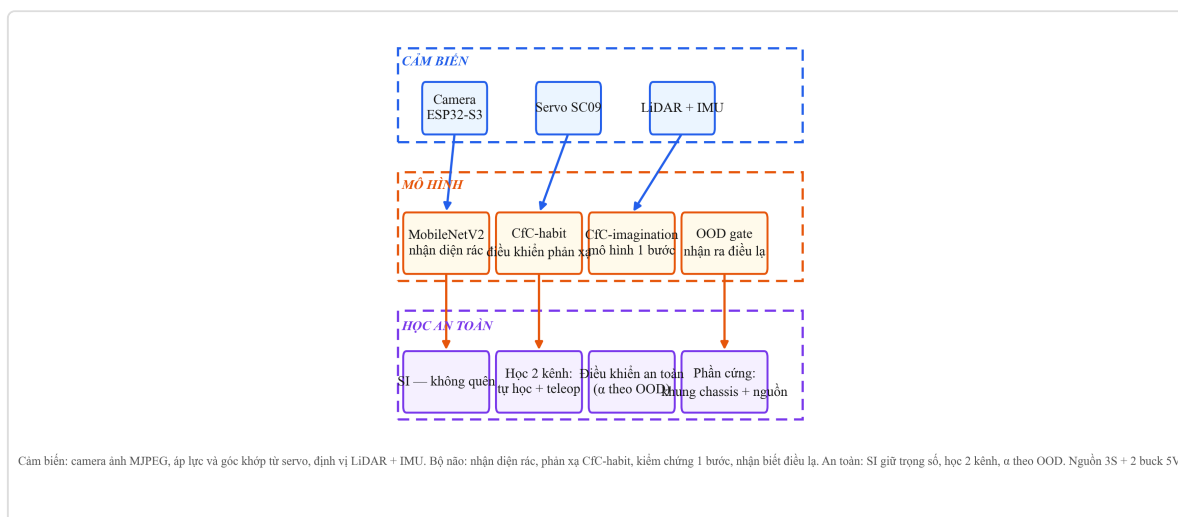
Nghiên cứu này đề xuất **Hybrid Mamba-2+Attention** — kiến trúc lai thay MLP (LeWM) bằng Mamba-2 trạng thái rời rạc trong JEPA predictor:

1. **Scheduled Sampling cho CfC** cải thiện chuỗi dự đoán $29\times$ — theo tra cứu, chưa có paper nào áp dụng cho ODE-RNN.
2. **Phát hiện SIGReg $\times$ ODE** — CfC giảm từ 78% (goal=25) xuống 6% (goal=100, $T=16$). Mamba-2 giảm ít hơn ($86\%\rightarrow 18\%$ cùng goal, dù dùng $T=4$ khác $T=16$), gợi ý trạng thái ODE khuếch đại SIGReg noise.
3. **Push-T beat LeWM AR trên cùng GPU**: Hybrid $94.7\%\pm 3.1\%$ (3 seeds) vs $88.0\%\pm 4.0\%$ (5090) / $86.0\%\pm 4.0\%$ (T4) — gap $+6.7\%/+8.7\%$.
4. **Hybrid Mamba-2 giảm sai số tích lũy so với AR ở $H=5$** : $94.7\%\pm 3.1\%$ vs $88.0\%\pm 4.0\%$. Ở $H=10,20$ cả hai giảm mạnh — xác nhận giới hạn chung của latent world model [2].

Hybrid Mamba-2+Attention là lựa chọn engineering đúng: trạng thái rời rạc giải quyết nhiễu $\times$ ODE mà vẫn giữ lợi thế thời gian so với AR không trạng thái. CEM planning chậm hơn LeWM $\sim 2.5\times$ (5090) do Mamba-2 kernel chưa tối ưu cho T4 — trade-off chấp nhận được để đổi lấy $+8.7\%$ performance.

7.2. Hướng nghiên cứu tiếp nối: robot nhặt rác di động

Kết quả của nghiên cứu mở ra hướng ứng dụng thực tế lớn hơn: **robot nhặt rác di động** — bài toán có ý nghĩa xã hội và môi trường, được dự kiến triển khai trong thời gian tới.



Hình H: Sơ đồ khối hệ thống robot nhặt rác di động — hướng nghiên cứu tiếp nối. Cảm biến: camera ảnh MJPEG, áp lực và góc khớp từ servo, định vị LiDAR + IMU. Bộ não: nhận diện rác, phản xạ CfC-habit, kiểm chứng 1 bước, nhận biết điều lạ (OOD). An toàn: SI giữ trọng số (không quên), học 2 kênh, α theo OOD. Nguồn: 2 cụm 3S, 2 buck 5V, L298N.

Bản thiết kế tận dụng ba kết quả chính của nghiên cứu: - **CfC-habit (phản xạ)** — học lệnh điều khiển từ người, tạo bộ điều khiển phản xạ nhanh thay cho CEM solver nặng (mô hình 16.6M tham số này đã học trên robot thật ở V0); - **CfC-imagination (mô hình 1 bước)** — đúc kết từ chính mô hình dự đoán thế giới trong nghiên cứu, dùng để "cuộn phim trước" kiểm chứng hành động; - **OOD gate (tự biết lạ)** — đưa phát hiện SIGReg×ODE vào ứng dụng thực tế: robot tự nhận biết trạng thái ngoài phân phối và điều chỉnh độ tin cậy.

Bản thiết kế này là **hướng nghiên cứu tiếp nối** — kêu gọi khai thác thực nghiệm trên nền robot thật, và thể hiện tính liên tục của hành trình nghiên cứu (từ V0 robot tay thật → V2.1 đạt chuẩn benchmark → V2.5.2 đẩy vào ứng dụng thực tế).

Cam kết tiến độ: nghiên cứu tiếp nối dự kiến hoàn thành trong **6 tuần** khi được hỗ trợ tiếp tục hoàn thiện.

8. Cách sử dụng, vận hành

8.1. [V0 — Robot thật] Vận hành phần cứng theo cử chỉ (demo)

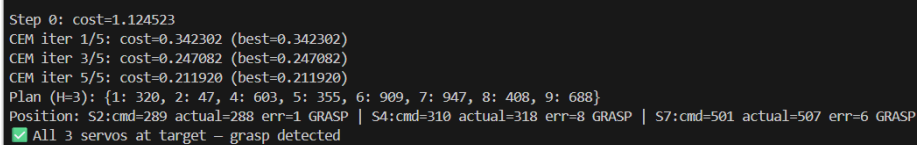
Chạy demo teleop điều khiển tay bionic theo cử chỉ bàn tay người:

```
python demo_grasp.py
```

8.1.2. [V0 — Robot thật] Vận hành world model + CEM trên robot thật

```
python robot_planner.py --goal <file> --model cfc --checkpoint <ckpt>
```

Chuỗi hoạt động: camera → encoder → CEM 300 chuỗi × 30 vòng → chọn action tối ưu → servo thực hiện → kiểm tra vị trí — grasp phát hiện thành công.



```
Step 0: cost=1.124523
CEM iter 1/5: cost=0.342302 (best=0.342302)
CEM iter 3/5: cost=0.247082 (best=0.247082)
CEM iter 5/5: cost=0.211920 (best=0.211920)
Plan (H=3): {1: 320, 2: 47, 4: 603, 5: 355, 6: 909, 7: 947, 8: 408, 9: 688}
Position: S2:cmd=289 actual=288 err=1 GRASP | S4:cmd=310 actual=318 err=8 GRASP | S7:cmd=501 actual=507 err=6 GRASP
✅ All 3 servos at target — grasp detected
```

Hình I: Log terminal thật — CEM planning chạy trên robot V0: Step 0 → CEM iter → Plan H=3 → báo GRASP

8.2. [V2.1 — Benchmark mô phỏng] Tái lập kết quả eval

```
python eval.py --config-name=pusht policy=<ckpt> --seed=3072 --config-name=pusht
```

Kết quả tái lập 3 seeds trên 2 GPU (T4/RTX 5090): 92%, 98%, 94% → 94.7% ± 3.1% (Push-T). Mã nguồn và checkpoint công khai:

Thành phần	Liên kết
 Mã nguồn	github.com/toan4965-ui/hybrid-mamba-2-attention-world-model
 Checkpoint	huggingface.co/hhian/checkpoints

Phân định rõ hai kết quả: Mục 8.1 là kết quả **[V0 — robot thật]** — tay bionic tự chế, dữ liệu tự thu nhỏ (8900 frame, 1 camera): đây là kiểm chứng nguyên lý (drift $34\times$ thấp hơn AR, grasp chạy được chuỗi CEM thật), **không phải kết quả thống kê tái lập được** vì dữ liệu nhỏ và chỉ trên 1 mô hình robot. Mục 8.2 là kết quả **[V2.1 — benchmark mô phỏng]** với dữ liệu chuẩn (20K episode), $3 \text{ seeds} \times 2 \text{ GPU}$ — tái lập được đầy đủ: $94.7\% \pm 3.1\%$ (Push-T).



Hình J: Ảnh thật robot đang grasp chai nước trong box thu dữ liệu

9. Tài liệu tham khảo

#	Nguồn
[1]	LeCun, Y. (2022). A Path Towards Autonomous Machine Intelligence.
[2]	Maes, L. et al. (2026). LeWorldModel: Stable End-to-End JEPA from Pixels. arXiv 2603.19312.
[3]	Hasani, R. et al. (2022). Closed-form Continuous-time Neural Networks. Nature Machine Intelligence.
[4]	Dao, T. & Gu, A. (2024). Transformers are SSMS: Generalized Models and Efficient Algorithms Through Structured State Space Duality. ICML 2024.
[5]	Balestriero, R. & LeCun, Y. (2025). LeJEPA: Provable and Scalable Self-Supervised Learning Without the Heuristics. arXiv 2511.08544.
[6]	Ma, C. & Najarian, K. (2025). Rethinking the long-range dependency in Mamba/SSM and transformer models. arXiv 2509.04226.
[7]	Huang, Y. (2026). VJEPA: Variational Joint Embedding Predictive Architectures as Probabilistic World Models. arXiv 2601.14354.
[8]	Gu, A. & Dao, T. (2023). Mamba: Linear-Time Sequence Modeling with Selective State Spaces. arXiv 2312.00752.
[9]	Lieber, O. et al. (2024). Jamba: A Hybrid Transformer-Mamba Language Model.
[10]	Li, Y. et al. (2026). TransMamba: A Sequence-Level Hybrid Transformer-Mamba Language Model. AAAI 2026.
[11]	Liu, X. et al. (2020). Neural SDE: Stabilizing Neural ODE Networks with Stochasticity. NeurIPS 2020.
[12]	Rob Knight (2022). DexHand V1.0: Open-Source Dexterous Humanoid Robot Hand. GitHub.
[13]	MambaLite-Micro (2025). Mamba LLM trước trên MCU với INT4 quantization. arXiv 2509.05488.
[14]	Quamba (2025). Post-training quantization to INT8 cho Mamba/SSM triển khai edge.